A CASE-STUDY BASED REPORT

ON

RARE DISEASE DETECTION USING DBSCAN-BASED ANOMALY DETECTION

submitted in partial fulfilment of the requirements for the Course

MACHINE LEARNING

BACHELOR OF TECHNOLOGY

in

A.Y.: 2026-27

by

Name :- SAMBAVARAM SARVESWARA REDDY	**Roll no:- 2520030049**
Name :- JAMPANI POORNESH KUMAR	**Roll no:- 2520030055**
Name :- KODEBOINA PRIYADARSHINI	**Roll no:- 2520030067**

Under the Esteemed guidance of

GADDAM RAVINDRA BABU

Assistant Professor, Dept. of CSE



(DEEMED TO BE UNIVERSITY)

Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

INDEX

Abstract
…… 3

1) Introduction 3-4
1.1) Background 3
1.2) Motivation for the Project 4
1.3) Overview of the Work Carried Out 4
1.4) Scope and Responsible-Use Statement 4

2) Problem Statement ..... ……4-5
2.1) Case-Study Scenario 4
2.2) Central Case-Study Question 4
2.3) Formal Problem Definition 5
2.4) Objectives that Guide the Study 5
2.5) Assumptions and Limitations 5
Literature Survey……...5-6

3) Dataset and Exploratory Data Analysis
……7-9
3.1) About the Dataset 7
3.2) Purpose of EDA 7
3.3) Preparation Steps Actually Performed 7-8
3.4) EDA Exploration - Implementation 8
3.5) Summary of EDA Findings 8
Methodology and System
Flow……9

4) Algorithm Selection and Implementation 9-12
4.1) Why DBSCAN 9
4.2)System Architecture 10
4.3) Preprocessing Pipeline 10
4.4) DBSCAN Parameter
Selection……10-11
4.5) Anomaly Identification 11
4.6) Web Application and API 11
4.7) Evaluation and Interpretation 11-12
4.8) Reproducibility and Execution 12

5) Results and Discussion……
……12-13

6) Responsible Use ,Risks and Limtations
……13
7) Conclusion and Future Enhancements……13

Signature of the Team members

ABSTRACT

Rare diseases and uncommon medical conditions can be difficult to identify because their presentations may be unusual and labelled examples may be limited. This case study examines a machine-learning application that uses Density-Based Spatial Clustering of Applications with Noise (DBSCAN) to identify medical profiles that differ from the dominant patterns in a dataset. The submitted project is a full-stack web application with a React/Vite frontend and a FastAPI backend. It accepts structured CSV data, inspects columns and missing values, allows the user to confirm identifier and target columns, imputes missing values, encodes categorical variables, standardizes features, recommends DBSCAN parameters using a k-distance curve, and marks records assigned to DBSCAN cluster −1 as potential anomalies.

The included demonstration dataset is synthetic and contains 400 records and 10 original columns. It includes seven numerical health-profile features, two categorical features, and a patient identifier. The raw file contains 48 missing cells and 0 duplicate rows. Reproducing the project's documented automatic preprocessing and DBSCAN parameter-selection procedure on this dataset yielded 4 non-noise clusters and 20 records marked as potential anomalies (5.0%). These are computational results on synthetic demonstration data, not clinical findings and not evidence of diagnostic accuracy.

The case study describes the data workflow, implementation, observed output, assumptions, and responsible-use constraints. The system is intended for educational exploration and research support only. An anomaly is a record that differs from the learned density structure; it is not a diagnosis of a rare disease. Any real-world use would require clinically validated datasets, expert review, privacy safeguards, and formal validation.

1. INTRODUCTION

1.1 Background

Machine learning can support exploratory analysis of complex datasets by discovering patterns that may not be obvious from individual records. In medical data, most observations may follow

common patterns while a small number of records have unusual combinations of measurements. Rare diseases can be underrepresented in labelled datasets, making it difficult to train a conventional supervised classifier for every condition.

DBSCAN is an unsupervised density-based clustering algorithm. It groups observations in dense regions of feature space and labels observations that do not belong to a sufficiently dense region as noise. In this project, noise points are surfaced as potential anomalies for inspection. This is a screening-oriented computational workflow, not a medical diagnostic method.

1.2 Motivation for the Project

The project is motivated by the need to explore unusual medical profiles without requiring a predefined number of clusters or a complete set of disease labels. The web interface makes the workflow accessible to users who want to upload a CSV, inspect its structure, configure DBSCAN, visualize clusters and anomalies, and download results.

1.3 Overview of the Work Carried Out

- Built a React 18 and Vite frontend with pages for upload, inspection, preprocessing, DBSCAN configuration, dashboard, anomalies, correlation analysis, confusion-matrix analysis, and patient-profile assessment.
- Implemented a FastAPI backend with REST endpoints for dataset upload, inspection, preprocessing, parameter selection, clustering, result retrieval, visualizations, correlation analysis, evaluation, and CSV download.
- Implemented a dataset-agnostic preprocessing pipeline using pandas, NumPy, scikit-learn imputers, one-hot encoding, and standard scaling.
- Implemented automatic DBSCAN parameter recommendations from a sorted k-distance curve and a rule-of-thumb MIN_SAMPLES setting.
- Included a synthetic demo dataset generated by the project's make_demo_data.py script; it is not real patient data.

1.4 Scope and Responsible-Use Statement

The scope is limited to anomaly exploration in structured tabular data. The output indicates unusualness relative to the uploaded dataset and selected features. It does not identify a specific disease, estimate a medically calibrated probability, replace a clinician, or establish that an outlier is ill. Users must not use the demo dataset to make healthcare decisions. For real patient data, appropriate authorization, de-identification, access control, secure storage, and institutional/ethical review are required.

2. PROBLEM STATEMENT

2.1 Case-Study Scenario

Consider a research team examining a table of patient-like measurements. The table contains common health-profile patterns and a small number of records with unusual combinations of measurements. The team has limited labels for rare conditions and wants to identify records that

warrant further review. The project provides a workflow for uploading the table, examining data quality, preprocessing features, clustering the records, and inspecting potential anomalies.

2.2 Central Case-Study Question

Can a density-based clustering workflow identify medical-profile records that are isolated from the dominant patterns in a structured dataset, while making the preprocessing and parameter choices visible to the user?

2.3 Formal Problem Definition

Let a dataset be represented by records $X = \{x_1, x_2, \ldots, x_n\}$, where each record is represented by a vector of cleaned, encoded, and scaled features. DBSCAN uses a neighbourhood radius ε (EPS) and a minimum number of points MIN_SAMPLES. It assigns records in sufficiently dense regions to clusters and assigns records that do not meet the density criteria to label -1. The project defines a potential anomaly as any record for which the DBSCAN label equals -1. This definition is algorithmic and relative to the dataset; it does not define disease status.

2.4 Objectives that Guide the Study

1. Provide a web interface for uploading a structured CSV dataset or loading the included demo data.
2. Inspect dataset dimensions, data types, missing values, duplicate rows, and possible identifier/target columns.
3. Prepare features through missing-value imputation, categorical encoding, and standardization.
4. Recommend DBSCAN parameters from the k-distance curve while allowing a manual parameter mode.
5. Identify noise points and summarize cluster counts, anomaly counts, and anomaly scores.
6. Present visual summaries, correlation exploration, optional evaluation views, and downloadable results.
7. State the limitations of anomaly detection clearly and prevent the output from being interpreted as a medical diagnosis.

2.5 Assumptions and Limitations

- Each row is assumed to represent one observation and each usable column a feature, unless the user confirms it is an identifier or target/label.
- Numerical values are meaningfully comparable after scaling; categorical variables can be represented with one-hot encoding.
- DBSCAN results depend strongly on feature selection, scaling, EPS, MIN_SAMPLES, and the density structure of the dataset.
- The demo data is synthetic and designed to contain common-like and unusual profiles. Its results cannot establish clinical validity.
- The backend stores one active dataset/model session in memory; the README describes it as a single-user demo. It is not designed as a production multi-user service.
- High-dimensional encoded data can make density-based distances less informative; rare but legitimate observations may be flagged and true abnormal cases may be missed.

LITERATURE SURVEY

The uploaded project documentation proposes a literature survey of recent work on clustering, anomaly detection, phenotype-based analysis and rare-disease research. The following summary preserves the studies and DOI strings as supplied in that document. The bibliographic details and DOI links should be checked against publisher records before final academic submission; they are not presented here as independently verified results.

Study 1 — DBSCAN and DBCV application to heterogeneous medical records for neuroblastoma (BioData Mining, 2023; DOI supplied: 10.1186/s13040-023-00245). The documentation describes the research gap as clustering heterogeneous medical records and reports that DBSCAN with DBCV identified clinically significant patient subgroups. Suggested future work is multi-centre validation.

Study 2 — Hypothesis generation for rare and undiagnosed diseases through clustering (PLOS ONE, 2024; DOI supplied: 10.1371/journal.pone.0309205). The documentation describes ontology clustering with protein-interaction networks to connect variants and phenotypes, with future integration into clinical decision support.

Study 3 — LS-BMO-HDBSCAN hybrid framework for efficient clustering (Scientific Reports, 2025; DOI supplied: 10.1038/s41598-025-24380-2). The documentation presents this as a hybrid clustering approach for noisy, high-dimensional data and recommends testing on biomedical datasets.

Study 4 — Labeled medical records corpus for rare disease detection (Scientific Reports, 2025; DOI string supplied in the documentation: 10.1038/s41598-025-2295). The documented research gap is limited annotated data; the suggested direction is to broaden coverage of rare conditions.

Study 5 — Few-shot learning for phenotype-driven diagnosis of rare genetic diseases (npj Digital Medicine, 2025; DOI supplied: 10.1038/s41746-025-00972). The documentation describes a few-shot phenotype-embedding approach intended to work with limited labelled examples, with possible EHR integration.

Additional topics listed in the supplied survey include phenotype-based AI pipelines, anomaly detection in electronic health records, genomic-variant clustering, DBSCAN/PCA for rare-disease subtyping, and autoencoder-plus-DBSCAN approaches for pediatric rare-disease screening. These entries provide background themes; claims about reported performance should be verified from the original papers.

Research direction in supplied survey	Method described	Relevance to this project
Heterogeneous medical records	DBSCAN + DBCV	Supports density-based clustering and cluster-validity analysis
Undiagnosed-disease hypothesis generation	Ontology clustering + protein interaction networks	Shows complementary biological knowledge approaches
Noisy/high-dimensional clustering	HDBSCAN hybrid optimization	Highlights limitations of density methods and parameter sensitivity

Rare-disease labelled corpora	Corpus creation + ML baselines	Emphasizes scarcity of reliable labelled data
Phenotype-driven diagnosis	Few-shot learning / embeddings	Alternative when limited labels are available
EHR anomaly screening	Isolation Forest + DBSCAN	Motivates baseline comparison, not proof of clinical accuracy

3. DATASET AND EXPLORATORY DATA ANALYSIS

3.1 About the Dataset

The ZIP contains sample_data/DEMO_synthetic_medical_profiles.csv. The file has 400 rows and 10 columns. The patient_id column is an artificial record identifier and should not be used as a clustering feature. The remaining columns describe age, systolic blood pressure, glucose, creatinine, hemoglobin, white blood cell count, BMI, sex, and smoking status. The dataset generator uses a fixed random seed and creates 380 common-pattern records and 20 deliberately unusual records before shuffling. These labels are part of the data-generation design and are not clinical ground truth.

Column	Type	Missing cells	Role / interpretation
patient_id	Categorical identifier	0	Record ID; exclude from clustering
age	Numerical	0	Age-like synthetic value
systolic_bp	Numerical	0	Synthetic systolic blood pressure
glucose	Numerical	12	Synthetic glucose measurement
creatinine	Numerical	12	Synthetic creatinine measurement
hemoglobin	Numerical	12	Synthetic hemoglobin measurement
wbc_count	Numerical	0	Synthetic white blood cell count
bmi	Numerical	0	Synthetic body-mass-index value
sex	Categorical	0	Synthetic category
smoker	Categorical	12	Synthetic smoking-status category

3.2 Purpose of EDA

Exploratory Data Analysis (EDA) is used to understand the dataset before clustering. It helps determine whether columns have suitable types, whether missing values need treatment, whether identifier or label columns could distort the analysis, whether there are duplicate rows, and whether feature distributions contain a wide range of scales. EDA is also a way to document data limitations before interpreting any model output.

3.3 Preparation Steps Actually Performed

1. Loaded the included CSV and inspected its shape, column names, data types, missingness, and duplicate rows.
2. Identified patient_id as a high-uniqueness identifier and excluded it from the feature matrix for the reproduced run.
3. Kept the remaining nine columns as input features: seven numerical columns and two categorical columns.
4. Imputed numerical missing values with the median and categorical missing values with the most frequent value.
5. One-hot encoded categorical features and standardized both numerical and encoded features using StandardScaler.
6. Generated a k-distance curve and selected automatic DBSCAN parameters using the project's implementation.

3.4 EDA Exploration – Implementation

The backend preprocessing module supports CSV encoding and delimiter detection, recognizes common missing-value tokens, converts numeric-looking text columns where appropriate, and drops fully empty columns. It suggests identifier and target columns based on names and value uniqueness; these are suggestions that the user confirms. During cleaning, confirmed IDs and target columns are excluded from unsupervised features. Constant columns, columns with more than 60% missing values, and certain high-cardinality free-text columns are removed. Remaining numerical values are median-imputed; categorical values are mode-imputed, one-hot encoded, and scaled.

EDA / quality check	Observed result in demo CSV
Dataset dimensions	400 rows × 10 original columns
Numerical features before encoding	7
Categorical features before encoding	2
Missing cells	48 across glucose, creatinine, hemoglobin, and smoker
Missingness by affected column	12 cells each in glucose, creatinine, hemoglobin, and smoker (3% per column)
Duplicate rows	0
Preprocessed matrix	400 rows × 11 encoded/scaled features
Identifier handling	patient_id excluded from model features

3.5 Summary of EDA Findings

The demo data is a small, structured, mixed-type dataset suitable for demonstrating preprocessing and density-based clustering. The 48 missing cells are sparse relative to the dataset size and are handled by the imputation pipeline. The numerical variables have different units and ranges, making standardization important before distance-based clustering. Categorical columns must be encoded before DBSCAN can process them. Since the file is synthetic, its distributions and planted unusual profiles are useful for demonstrating the software pipeline but should not be treated as representative of a clinical population.

METHODOLOGY AND SYSTEM FLOW

The project documentation gives the conceptual flow as: Authorised Dataset → Data Inspection → Data Cleaning → Missing-Value Treatment → Feature Selection → Categorical Encoding → Numerical Scaling → DBSCAN Model → Hyperparameter Exploration → Anomaly Detection → PCA Visualization → Evaluation → Baseline Comparison → Final Screening Result.

Implementation evidence from the ZIP is more specific: the runnable application is a React 18 + Vite frontend that calls a Python FastAPI backend. The backend uses pandas, NumPy and scikit-learn. The project documentation mentions Streamlit, Matplotlib/Seaborn, OpenPyXL and Joblib as an experimental setup, but the supplied ZIP's README and package files document React/Vite and FastAPI; therefore this report treats the ZIP as the implementation source and the project document as the conceptual/academic methodology.

Data handling begins with CSV upload or the synthetic demo file. The inspection screen reports shape, columns, data types, missingness and duplicate rows and suggests ID/label columns for the user to confirm. Confirmed identifiers and labels are excluded from clustering features to reduce leakage. The preprocessing code handles common missing tokens, converts numeric-looking text when possible, removes unsuitable columns, imputes missing values, one-hot encodes categories and standardizes features.

DBSCAN is fitted on the full processed feature matrix. The implementation selects a recommended MIN_SAMPLES using a bounded rule of thumb and estimates EPS from a sorted k-distance curve; a fallback percentile is used when the detected elbow is too early. Manual parameter entry is also supported. The DBSCAN noise label −1 becomes the Potential Anomaly flag. PCA is used for visualization, not as the input that defines the anomaly label.

Evaluation views can compare DBSCAN outputs against available ground-truth labels, selected clinical criteria or an ensemble/Isolation Forest consensus benchmark. Metrics such as precision, recall, F1, sensitivity and specificity are meaningful only when the comparison reference is valid. Consensus from another algorithm is not independent clinical ground truth.

4. ALGORITHM SELECTION AND IMPLEMENTATION

4.1 Why DBSCAN

DBSCAN was selected because it discovers dense groups without requiring the number of clusters to be specified in advance and explicitly marks points that do not fit density-based clusters as noise. This matches the project's goal of surfacing unusual profiles in an unsupervised setting. However, DBSCAN is sensitive to EPS and MIN_SAMPLES, and it can struggle when clusters have very different densities or when the feature space becomes high-dimensional.

4.2 System Architecture

Layer	Technology / responsibility
Frontend	React 18, React Router, Vite, Recharts; upload flow, inspection, configuration, dashboards and visualizations
REST API	FastAPI; receives uploads and requests, validates inputs, coordinates preprocessing and model execution
Data handling	pandas and NumPy; CSV loading, inspection, missing-value handling and result tables
Machine learning	scikit-learn; SimpleImputer, OneHotEncoder, StandardScaler, NearestNeighbors, DBSCAN, PCA and evaluation utilities
Output	In-memory result state and downloadable CSV with cluster/anomaly fields

The frontend communicates with the backend through HTTP requests. The backend exposes the API under the /api prefix and enables CORS for local frontend development origins. The current implementation maintains a single in-memory session; uploaded dataset and model state are not a persistent database.

4.3 Preprocessing Pipeline

1. CSV loading: reads the uploaded file, normalizes column names and common missing tokens, and rejects files with fewer than 10 rows or fewer than two columns.
2. Column inspection: summarizes rows, columns, types, missing values, uniqueness, and duplicate rows; suggests ID and target columns.
3. Feature selection: excludes confirmed ID/target columns and removes unusable columns according to the implemented cleaning rules.
4. Imputation: uses the median for numerical features and the most frequent category for categorical features.
5. Encoding: converts categorical features to one-hot encoded columns.
6. Scaling: applies standard scaling so input dimensions are more comparable for distance calculations.

7. Parameter recommendation: calculates k-nearest-neighbour distances and derives an EPS suggestion from the sorted k-distance curve.

4.4 DBSCAN Parameter Selection

The implementation estimates MIN_SAMPLES using a rule of thumb bounded by the number of rows and dimensions: approximately twice the number of dimensions, capped at 15, with a minimum of 4 for typical datasets. It calculates each observation's distance to its k-th nearest neighbour, sorts these distances, and detects an elbow. If the detected elbow is too early in the curve, the code falls back to the 92nd percentile. The user can also select manual mode and provide EPS and MIN_SAMPLES.

For the included demo dataset, the reproduced automatic run produced 11 encoded/scaled features, MIN_SAMPLES = 15, EPS ≈ 2.4012, and parameter-selection method 'k-distance elbow'. These values are dataset- and implementation-specific; they may differ if the dataset, package versions, or settings change.

4.5 Anomaly Identification

After fitting scikit-learn DBSCAN, records with cluster label −1 are marked Potential_Anomaly = True and Anomaly_Status = 'Potential Anomaly'. Other records are marked 'Within Main Pattern'. The output also includes DBSCAN_Cluster and Anomaly_Score. In the implementation, Anomaly_Score is the k-th-neighbour distance in scaled feature space, not a calibrated disease probability. A higher value indicates greater isolation under this representation.

4.6 Web Application and API

The application is designed as a guided workflow: upload a CSV or load demo data; inspect the dataset and confirm ID/target columns; preprocess; review the k-distance recommendation; run DBSCAN; inspect dashboard charts and anomaly records; review correlation and evaluation views; optionally assess a new profile in the fitted feature space; and download the output CSV.

Endpoint	Purpose
POST /api/upload	Upload and parse a CSV file
POST /api/upload-sample	Load the included synthetic demo dataset
POST /api/analyze	Return dataset inspection and EDA summaries
POST /api/preprocess	Clean, impute, encode, scale and recommend parameters
GET /api/kdistance	Return k-distance curve and parameter recommendation
POST /api/dbscan	Run DBSCAN in automatic or manual mode
GET /api/results	Return model summary
GET /api/visualization	Return visualization data
GET /api/correlation	Return feature-correlation analysis
GET /api/confusion-matrix	Return evaluation analysis when suitable reference/benchmark information is available
GET /api/anomalies	Return potential anomaly records
POST /api/assess-patient	Compare a new feature profile with the fitted data patterns

GET /api/download	Download CSV results
POST /api/reset	Clear the active in-memory session

4.7 Evaluation and Interpretation

The code includes a confusion-matrix analysis view that can compare anomaly predictions with available labels, selected clinical criteria, or an Isolation Forest consensus benchmark. Metrics may include sensitivity/recall, specificity, precision, negative predictive value, false-positive rate, false-negative rate, accuracy, balanced accuracy and F1 score. These metrics must be interpreted according to the selected reference source. If the comparison labels are generated by another algorithm, they are not independent clinical ground truth. This report does not claim clinically validated accuracy.

4.8 Reproducibility and Execution

1. Use Python 3.10 or later and Node.js 18 or later.
2. Open a terminal in the extracted project folder and enter the backend directory.
3. Create and activate a virtual environment, then install backend/requirements.txt.
4. Start the API using `uvicorn main:app --reload --port 8000` from the backend directory; inspect `http://localhost:8000/docs`.
5. Open a second terminal in the frontend directory, run `npm install`, then `npm run dev`.
6. Open the local URL printed by Vite, usually `http://localhost:5173`, and select Load DEMO dataset to follow the complete analysis flow.
7. Review the results and download the CSV. Use only synthetic or appropriately authorized data.

5. RESULTS AND DISCUSSION

On the supplied synthetic demo CSV, the documented preprocessing and automatic DBSCAN configuration produced 4 non-noise clusters and 20 potential anomalies out of 400 records. The remaining 380 records were assigned to non-noise clusters. The resulting potential-anomaly rate was 5.0%. The observed non-noise cluster sizes were: cluster 0: 30 records, cluster 1: 146 records, cluster 2: 47 records, cluster 3: 157 records.

This output is consistent with the purpose of a demonstration dataset that includes a small group of deliberately unusual profiles. It demonstrates that the preprocessing and clustering pipeline can separate some isolated observations from denser regions. It does not establish that the identified records correspond to rare diseases, nor does the anomaly percentage represent the prevalence of any condition. The score and labels are dependent on feature encoding, scaling, parameter selection and the dataset itself.

Result item	**Observed demo run**
Input records	400
Original columns	10
Features after preprocessing	11
Automatic EPS	2.4012
Automatic MIN_SAMPLES	15

Non-noise clusters	4
Potential anomalies (cluster −1)	20
Records in non-noise clusters	380
Potential-anomaly rate	5.0%

RESULTS INTERPRETATION NOTE

The project documentation includes an illustrative comparison table with accuracy, precision, recall, F1 and ROC-AUC values, and an illustrative statement that DBSCAN found approximately 18 anomalies (9%) while Isolation Forest found approximately 20 (10%). These figures are not linked in the document to a clearly specified run, dataset split or traceable result file. They have therefore not been treated as measured results for this implementation. The results section of this report instead reports the reproduced DBSCAN run on the ZIP's 400-row synthetic demo dataset and explicitly labels it as demonstration-only.

6. RESPONSIBLE USE, RISKS AND LIMITATIONS

- Not a diagnostic tool: anomaly detection identifies statistical unusualness, not disease. Clinical interpretation must be performed by qualified professionals.
- Synthetic dataset: the included CSV is generated by code and is not a real clinical dataset. Results cannot be generalized to patients or healthcare populations.
- False positives and false negatives: a healthy person may be an outlier, and a rare-disease case may appear in a common cluster.
- Parameter and feature sensitivity: EPS, MIN_SAMPLES, scaling, selected columns, missing-value imputation and categorical encoding affect outcomes.
- High dimensionality: one-hot encoding may increase dimensionality and weaken the usefulness of distance-based density measures.
- Validation: meaningful clinical evaluation requires representative, consented data, a defensible reference standard, subgroup analysis, external validation and review by domain experts.
- Privacy and security: the current demo keeps one session in memory and should not be deployed with identifiable health information without a proper security architecture.
- Fairness and data quality: missingness, measurement practices, sampling bias and population differences can influence which records are flagged.
- Patient assessment: a dataset-similarity result or profile factor is not a medical risk score or diagnosis.

7. CONCLUSION AND FUTURE ENHANCEMENTS

This case study documents a full-stack application for exploring potential anomalies in structured medical-profile data using DBSCAN. The project combines a React/Vite interface, FastAPI REST endpoints, data inspection and preprocessing, k-distance-based parameter recommendations, clustering, visual summaries, correlation analysis, optional evaluation views and CSV export. On the included synthetic dataset, the reproduced automatic run identified 20 potential anomalies and

four non-noise clusters. These results demonstrate the software workflow only and must not be interpreted as clinical evidence.

Future enhancements include testing with ethically approved and clinically curated datasets; adding robust validation and uncertainty reporting; supporting multiple concurrent users with persistent, access-controlled sessions; recording experiment settings and dataset provenance; adding sensitivity analysis across EPS and MIN_SAMPLES; improving model monitoring and audit trails; and involving clinicians in reviewing whether flagged patterns are clinically meaningful. Any future clinical-facing version should undergo appropriate institutional, regulatory, privacy and safety review before use.

REFERENCES

1. Project source code and README included in the supplied ZIP: Rare Disease Detection Using DBSCAN-Based Anomaly Detection (React/Vite frontend and FastAPI/scikit-learn backend).
2. Ester, M., Kriegel, H.-P., Sander, J., & Xu, X. (1996). A density-based algorithm for discovering clusters in large spatial databases with noise. Proceedings of the Second International Conference on Knowledge Discovery and Data Mining, 226–231.
3. scikit-learn developers. DBSCAN documentation. https://scikit-learn.org/stable/modules/generated/sklearn.cluster.DBSCAN.html
4. scikit-learn developers. Preprocessing data documentation. https://scikit-learn.org/stable/modules/preprocessing.html
5. pandas documentation. https://pandas.pydata.org/docs/
6. FastAPI documentation. https://fastapi.tiangolo.com/
7. React documentation. https://react.dev/
8. Vite documentation. https://vite.dev/

Guide Signature Course Instructor